

CSCE 627 THEORY OF COMPUTABILITY · MODULE 04 · LECTURE NOTES

Turing Machines

The model, and why it is the right one.

MODULE OUTCOMES

- Define a Turing machine and trace a computation.
- Explain robustness — why variants add no power.
- Distinguish decidable from Turing-recognisable.
- Explain the universal Turing machine and what it established.
- Encode machines as strings and explain why that is essential.

1 The model and its robustness

A TURING MACHINE IS A FINITE AUTOMATON WITH UNBOUNDED, REWRITABLE MEMORY

An infinite tape, a head that reads and writes a single cell and moves one step left or right, and a finite control. That is the entire model, and its simplicity is deliberate.

The finite control is the program and the tape is the memory — so the machine has a *fixed-size description* and *unbounded working space*, which is exactly the situation of a real computer and is what makes the model a reasonable idealisation rather than an arbitrary one.

Turing designed it to model a person computing with paper and pencil, and section 9 of his 1936 paper is an argument about what such a person can do — finitely many states of mind, a bounded number of symbols attended to at once, local changes. **That argument, rather than any mathematical result, is why the Church–Turing thesis is credible** (Module 01 §2).

And the model is deliberately inefficient. It is a tool for proving what is possible, not for computing — which is why its *polynomial-time* class is meaningful (CSCE 637's subject, since polynomial-time is robust across models) and its constants and small-degree polynomials are not.

MULTIPLE TAPES → simulate on one tape with the tracks interleaved. Quadratic slowdown. SAME CLASS.
 TWO-WAY INFINITE TAPE → fold it in half and store both halves interleaved on a one-way tape. SAME CLASS.
 NON-DETERMINISM → breadth-first search the computation tree on a deterministic machine. EXPONENTIAL slowdown. SAME CLASS.
 RANDOM ACCESS → simulate addressing by scanning for the address. Polynomial slowdown. SAME CLASS.
 A STACK INSTEAD OF A TAPE → NOT the same class. That is Module 03, and it is strictly weaker.
 TWO STACKS → same class as one tape, which is a pleasant surprise (one stack for each side of the head).
 TWO COUNTERS → ALSO the same class (Minsky), with only increment, decrement and test-for-zero. Astonishing, and true.

SO THE CLASS OF COMPUTABLE FUNCTIONS IS ROBUST under every reasonable change to the model, and only the TIME changes.

WHICH IS WHY THIS COURSE AND CSCE 637 DIVIDE WHERE THEY DO: computability is model-independent, so it can be settled once; complexity is model-sensitive, so it needs care about which model -- and the NON-DETERMINISM row above is exactly where P vs NP lives.

Robustness is what makes the definition trustworthy. A notion of computability that changed when you added a second tape would not be worth proving theorems about, because the theorems would be about the formalism rather than about computation. **The fact that it does not change is the second pillar of the Church–Turing thesis**, alongside Module 01's independent-formalisation argument.

2 Decidable and recognisable

	Decidable	Turing-recognisable
On an input in the language	Halts and accepts.	Halts and accepts.
On an input not in the language	Halts and rejects.	May run forever without ever answering.
Also called	Recursive; total; solvable.	Semi-decidable; recursively enumerable; computably enumerable.
Closed under complement	Yes — swap the answers.	NO — and that asymmetry is the key to the whole subject.
Examples	Primality; membership in a regular or context-free language.	Halting; provability in a formal system; whether a Diophantine equation has a solution.

The asymmetry is the whole subject, and one theorem carries most of the weight: if a language and its complement are both Turing-recognisable, then the language is decidable. The proof is one line — run both recognisers in parallel, and since every input is in one or the other, one of them must eventually halt and accept, which tells you the answer. **Module 05 uses this to deduce that the complement of the halting problem is not even recognisable,** and Module 07 builds the whole hierarchy on it.

RECOGNISABLE MEANS 'VERIFIABLE WHEN THE ANSWER IS YES'

A language is recognisable exactly when its members have finite certificates that can be checked.

'This program halts on this input' is witnessed by the halting computation itself — a finite object you can verify step by step.

So recognisable is the *enumerable* class: you can write a program that lists the members, in some order, eventually printing every one — **and you can never tell whether a particular item that has not appeared is absent or merely late.** That is precisely the content of 'may run forever'.

Which is why it is also called recursively enumerable, and the enumeration view is frequently the easier one for proofs — showing a language is recognisable by exhibiting an enumerator is often simpler than designing a recogniser.

And the parallel with CSCE 637's NP is exact in form: NP is 'verifiable in polynomial time given a short certificate'; recognisable is 'verifiable eventually, given any finite certificate' — the same structure with the resource bound removed. **Which is why the two courses' central distinctions feel similar, and why 'is checking easier than finding' is the question in both.**

3 Universality and encoding

THE UNIVERSAL MACHINE IS THE IDEA THAT MADE COMPUTERS POSSIBLE

Encode a Turing machine as a string — its states, alphabet, and transition table are a finite object, so this is routine. **Then there is a single machine U which, given the encoding of M together with an input w, simulates M on w.**

So one fixed machine can do whatever any machine does, given that machine's description as data — which is the stored-program computer, stated and proved as a theorem in 1936, a decade before one was built. The architecture followed the theorem rather than the reverse.

The encoding is what makes it possible, and it is also what makes Module 05 possible: **once programs are data, a program can be given its own description as input**, which is where diagonalisation comes from and is not a trick.

And it means the halting problem is a question about a *language* — the set of encoded pairs (M, w) such that M halts on w — **which is what lets the mathematics of Modules 02 and 03 apply to questions about programs.** Without the encoding there would be nothing to prove theorems about.

A Turing machine is a finite object: a state set, an alphabet, and a transition table. So it can be written down as a string.

THREE CONSEQUENCES, EACH ESSENTIAL TO WHAT FOLLOWS:

1. THE UNIVERSAL MACHINE exists -- one machine simulating any other from its description. The stored-program computer, proved before it was built.
2. SELF-REFERENCE becomes available. A machine can be given ITS OWN description as input. This is not a paradox and not a trick: it is a string being passed to a program, which happens every time you run a compiler on its own source.
-- Module 05's diagonalisation and Module 09's recursion theorem both come from here.
3. PROPERTIES OF PROGRAMS BECOME LANGUAGES. "Does M halt on w?" is membership in a set of strings, so the entire apparatus of Modules 02 and 03 applies to questions ABOUT PROGRAMS.
-- Which is what makes Module 06's reductions possible, and Module 08's results about real tools meaningful.

ALL THREE DEPEND ON ONE OBSERVATION: a program is a finite object, so it is a string. Everything after this module rests on it.

4 Counting, and what it does not give you

ALMOST ALL FUNCTIONS ARE UNCOMPUTABLE, BY COUNTING

There are countably many Turing machines, because each is describable by a finite string over a finite alphabet, and the finite strings over a finite alphabet are countable.

And there are uncountably many functions from strings to $\{0,1\}$, by Cantor's diagonal argument — there are as many as there are subsets of a countable set.

So almost every such function is computed by no machine whatsoever. The computable functions are a vanishingly small fragment of all functions, and this takes two lines to establish.

Which proves that uncomputable functions exist and names none of them. Module 05's contribution is a *specific, natural, and practically important uncomputable problem* — and that is much harder and much more useful than this counting argument. The distinction is worth holding: an existence proof by counting tells you nothing about which problems are affected, and the whole value of Modules 05 through 08 is that they identify the problems you actually care about.

- **A simple model computes everything computable**, and every reasonable variant computes the same class (§1).
- **So the class is robust**, which is what makes impossibility proofs about it worth having rather than being artefacts of a formalism.
- **Decidable and recognisable are genuinely different**, and **the complement asymmetry between them is Module 05's main tool** (§2).
- **Programs are data, so programs can be inputs** — which makes self-reference and the universal machine both available, and both are needed (§3).
- **And uncomputable functions exist for the price of a counting argument**, while **finding an *interesting* one is Module 05's job**. If one thing from this module should stick, it is that **a program is a finite object and therefore a string** — everything from here uses it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — chapter 3	The machine, the variants of §1, and the decidable/recognisable distinction of §2.
Turing — On Computable Numbers, sections 1–7 and 9 (free)	The model and the universal machine of §3, in the original , plus the section 9 argument that justifies the whole enterprise.
Minsky — Computation: Finite and Infinite Machines	§1's two-counter result , and the clearest treatment of the minimal models. Library copy.
MIT 18.404J — lectures 5–7 (free)	This module lectured , with the encoding discussion of §3 made careful.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a Turing machine simulator** and run a machine that decides $a^n b^n$.
- 2 **Write a machine that adds two binary numbers** and trace it on a small input.
- 3 **Implement a two-tape machine and simulate it on one tape**, and measure the slowdown.
- 4 **Simulate a non-deterministic machine deterministically** and report the blow-up.
- 5 **Design an encoding of Turing machines as strings** and write a decoder.
- 6 **Implement a universal machine** that runs an encoded machine on an encoded input.
- 7 **Feed a machine its own encoding** and confirm nothing paradoxical happens.
- 8 **Give an example of a recognisable language that is not decidable**, and say why your recogniser may not halt.
- 9 **Prove the complement theorem of §2** in your own words.
- 10 **Write out the counting argument** and state exactly what it does and does not establish.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define a Turing machine and say what Turing designed it to model.
- 2 Name six variants and whether each changes the class.
- 3 Which row is P versus NP, and why does that divide the courses?
- 4 Distinguish decidable from recognisable on five axes.
- 5 State and prove the complement theorem.
- 6 What does recognisable mean intuitively, and what is the NP parallel?
- 7 What is the universal machine, and what did it establish?
- 8 Give the three consequences of encoding programs as strings.
- 9 Give the counting argument and say what it fails to provide.